

Go 服务端云部署指南

一、部署方案总览与推荐

本文档为 `hertz-track` Go 服务端项目提供两种主流的云部署方案：**Docker Compose** 和 **Binary + systemd**。



方案一：Docker Compose（推荐）

优点：

- **环境一致性**：将应用与依赖（如 MongoDB）打包成可移植的镜像，保证开发、测试、生产环境完全一致。
- **快速部署**：通过 `docker compose up` 命令一键启动整个服务栈，极大简化了部署流程。
- **易于管理**：服务启停、更新、回滚、扩缩容（虽然本项目是单体）都可通过 Compose 命令完成。
- **隔离性好**：每个服务运行在独立的容器中，网络和存储相互隔离，安全性更高。

适用场景：

- 推荐所有环境（开发、测试、生产）优先使用此方案。
- 特别适合需要快速迭代、持续集成/持续部署（CI/CD）的团队。



方案二：Binary + systemd（次选）

优点：

- **性能原生**：二进制文件直接在宿主机运行，无容器化带来的性能损耗（尽管通常可忽略不计）。
- **资源占用低**：相比运行 Docker 引擎和多个容器，此方案对系统资源的占用更少。
- **传统且成熟**：对于熟悉传统运维方式的团队，此方案更易于理解和排查问题。

适用场景：

- 对资源占用极其敏感的边缘计算或小型服务器。
- 团队技术栈不熟悉 Docker，或已有成熟的传统部署流程。
- 无复杂依赖，或依赖已由其他方式独立管理（如使用云数据库）。

结论：除非有特殊约束，否则 **Docker Compose 是首选方案**，它能显著提升部署效率与可靠性。

二、准备条件

在开始部署之前，请确保您已满足以下条件：

1. 服务器环境

- **操作系统**：推荐 Ubuntu 22.04 LTS (Jammy Jellyfish) 或其他主流 Linux 发行版。
- **硬件**：至少 1 核 CPU / 1GB RAM / 10GB 磁盘空间。
- **软件**：
 - `sudo` 权限的用户账户。
 - **Docker & Docker Compose** (若选择方案一)。
 - **Nginx** (若选择方案二，或方案一也希望在容器前加一层反代)。

2. 域名与网络

- **域名**：准备一个域名（如 `track.example.com`）并将其 DNS A 记录指向您的服务器公网 IP。
- **防火墙**：确保服务器防火墙（如 `ufw`）或云服务商的安全组已开放以下端口：
 - `22` (SSH)
 - `80` (HTTP)
 - `443` (HTTPS)
 - `8080` (应用端口，可按需调整，建议不对公网暴露，仅由 Nginx 访问)

3. 数据库

- **MongoDB**：应用需要连接 MongoDB 数据库。您可以选择以下任一方式：
 - **Docker Compose 自建**：`docker-compose.yml` 已包含一个 `mongo` 服务，开箱即用。
 - **服务器自建**：在服务器上独立安装和运行 MongoDB 服务。
 - **云数据库**：使用 MongoDB Atlas, DocumentDB 等托管服务。

4. 密钥管理



核心原则：绝不硬编码任何凭证！

所有敏感信息，如数据库连接字符串、API 密钥、微信 App Secret 等，都必须通过**环境变量**注入。本项目提供 `.env.example` 文件作为模板，实际部署时应创建 `.env` 文件（或等效的 `systemd` 环境变量文件）并填写真实值。该文件**绝不能**提交到 Git 仓库。

四、方案二：使用 Binary + systemd 部署

此方案是一种更传统的部署方式，涉及在目标服务器上直接运行编译好的二进制文件，并使用 `systemd` 作为服务管理器来保证其稳定运行，同时配置 `Nginx` 作为反向代理。

1. 流程概述

★ 部署流程概览

1. **交叉编译**: 在本地或 CI 环境中, 将 Go 源码编译成目标服务器架构 (如 `linux/amd64`) 的二进制文件。
2. **上传文件**: 使用 `scp` 将二进制文件、`.env` 配置文件和 `systemd` 服务单元文件上传到服务器。
3. **安装服务**: 在服务器上, 将文件移动到指定位置, 并设置好权限。
4. **配置 Nginx**: 设置 Nginx 将来自公网的请求反向代理到本地运行的应用端口。
5. **启动与管理**: 使用 `systemctl` 命令重载配置、启动并管理应用服务。

2. 部署产物说明

所有相关文件都位于 `server/deploy/` 目录下。

1) `systemd/hertz-track.service`

这是 `systemd` 的服务单元文件, 定义了如何管理我们的应用。

代码块

```
1 [Unit]
2 Description=Hertz Track HTTP API Service
3 After=network-online.target
4
5 [Service]
6 Type=simple
7 User=www-data
8 WorkingDirectory=/opt/hertz-track
9 ExecStart=/opt/hertz-track/hertz-track
10 Restart=always
11 EnvironmentFile=/etc/hertz-track.env
12
13 [Install]
14 WantedBy=multi-user.target
15
```

- **User**: 建议使用一个低权限用户 (如 `www-data`) 运行服务。
- **WorkingDirectory**: 服务的工作目录。
- **ExecStart**: 启动服务的命令。
- **Restart=always**: 当服务意外退出时, `systemd` 会自动重启它。
- **EnvironmentFile**: 指定环境变量文件的路径, 所有配置从这里读取。

2) nginx/hertz-track.conf

这是 Nginx 的配置文件示例，用于设置反向代理。

代码块

```
1 upstream hertz_track_backend {
2     server 127.0.0.1:8080;
3     keepalive 32;
4 }
5
6 server {
7     listen 80;
8     server_name track.example.com; # 替换为你的域名
9
10    location / {
11        proxy_pass http://hertz_track_backend;
12        # ... 其他代理头设置 ...
13    }
14 }
15
```

此配置将 `track.example.com` 的 80 端口流量转发到本地的 8080 端口。

3) scripts/deploy_binary.sh

提供了一个完整的自动化部署脚本示例，展示了从编译到远程启动的全过程。**请注意，使用前必须修改其中的占位符**（`REMOTE_USER`，`REMOTE_HOST` 等）。

3. 手动部署步骤

1. 交叉编译：在本地 `server` 目录下执行。

代码块

```
1 GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o build/hertz-track
  ./cmd/server
2
```

2. 准备 `.env` 文件：基于 `.env.example` 创建一个 `.env` 文件，并填入真实的生产环境配置，特别是数据库连接串。

代码块

```
1 # server/.env for production
2 MONGO_URI=mongodb://user:password@your_mongo_host:27017/trackapp
3 SERVER_ADDR=:8080
```

```
4 # ... 其他配置
5
```

3. 上传文件到服务器 (替换 `user@host` 为你的服务器地址):

代码块

```
1 scp build/hertz-track user@host:/tmp/
2 scp .env user@host:/tmp/hertz-track.env
3 scp deploy/systemd/hertz-track.service user@host:/tmp/
4
```

4. 在服务器上安装:

代码块

```
1 # 创建用户和目录
2 sudo useradd -r -s /bin/false www-data
3 sudo mkdir -p /opt/hertz-track
4
5 # 移动文件并设置权限
6 sudo mv /tmp/hertz-track /opt/hertz-track/
7 sudo mv /tmp/hertz-track.env /etc/
8 sudo mv /tmp/hertz-track.service /etc/systemd/system/
9
10 sudo chown -R www-data:www-data /opt/hertz-track
11 sudo chmod 755 /opt/hertz-track/hertz-track
12 sudo chmod 640 /etc/hertz-track.env
13 sudo chown root:www-data /etc/hertz-track.env
14
```

5. 启动服务:

代码块

```
1 sudo systemctl daemon-reload
2 sudo systemctl enable --now hertz-track.service
3 sudo systemctl status hertz-track.service
4
```

6. 配置 Nginx:

- 将 `deploy/nginx/hertz-track.conf` 上传到服务器的 `/etc/nginx/sites-available/`。

- 创建软链接: `sudo ln -s /etc/nginx/sites-available/hertz-track.conf /etc/nginx/sites-enabled/`
- 测试并重载 Nginx: `sudo nginx -t && sudo systemctl reload nginx`

4. 配置 TLS (HTTPS)

强烈建议为您的服务启用 HTTPS。使用 [Certbot](#) 可以轻松获取免费的 Let's Encrypt 证书。

1. 安装 Certbot:

代码块

```
1 sudo apt update && sudo apt install certbot python3-certbot-nginx
2
```

2. 获取并安装证书:

代码块

```
1 sudo certbot --nginx -d track.example.com
2
```

Certbot 会自动修改您的 Nginx 配置以启用 HTTPS，并设置定时任务自动续期证书。

五、运行与健康检查

部署完成后，需要验证服务是否正常运行。

1. 访问应用

- **通过 Nginx/Docker:** 在浏览器中访问您的域名 `http://track.example.com` 或配置的 IP 与端口。
- **健康检查路由:** Hertz 默认在 `/healthz` 路径提供了一个健康检查端点。您可以 `curl http://localhost:8080/healthz` (在服务器或容器内) 来确认服务是否启动。

2. 查看日志

- **Docker Compose:** `cd server/deploy && docker compose logs -f api`
- **systemd:** `sudo journalctl -u hertz-track -f -n 50`

3. 排错清单

常见问题排查

1. 服务无法启动:

- 检查日志，看是否有 `panic` 或明显的错误信息（如“address already in use”端口占用）。
- 确认 `.env` 或环境变量文件中的配置是否正确，特别是 `MONGO_URI`。

2. 502 Bad Gateway:

- Nginx 无法连接到后端的 Hertz 服务。
- 确认 Hertz 服务是否在运行，并且监听在 Nginx `proxy_pass` 指向的地址（`127.0.0.1:8080`）。
- 检查服务器防火墙是否阻止了本地环回（loopback）连接。

3. 数据库连接失败:

- 确认 `MONGO_URI` 中的地址、端口、用户名、密码和数据库名是否正确。
- 如果使用 Docker Compose，确保 `api` 和 `mongo` 容器在同一个网络下。
- 检查网络策略或防火墙是否允许应用服务器访问数据库服务器。

六、安全与合规

- 禁止硬编码凭证：**所有密钥、密码和 Token 都必须通过环境变量注入，绝不写入代码或 `Dockerfile`。
- 最小权限原则：**
 - 运行服务的用户（如 `www-data`）应为低权限用户。
 - MongoDB 用户也应遵循最小权限原则，仅授予应用所需的读写权限。
- 使用 `distroless` 镜像：**此镜像不含 shell 或其他工具，能有效减少被攻击的风险。
- 只读根文件系统：**对于 `distroless` 镜像，可以进一步在 `docker-compose.yml` 中设置 `read_only: true`，并将需要写入的目录（如 `/tmp`）挂载为 `tmpfs`，增强安全性。
- 最小化端口暴露：**只将必要的端口（如 `80`，`443`）暴露到公网。应用端口 `8080` 和数据库端口 `27017` 应仅限内部网络访问。

七、附录：文件路径索引

文件/目录	路径	说明
Dockerfile	<code>server/deploy/Dockerfile</code>	用于构建应用 Docker 镜像。
Docker Compose	<code>server/deploy/docker-compose.yml</code>	编排 <code>api</code> 和 <code>mongo</code> 服务。
Compose 环境变量模板		

	<code>server/deploy/.env.example</code>	<code>docker-compose</code> 使用的环境变量模板。
systemd Unit	<code>server/deploy/systemd/hertz-track.service</code>	二进制部署的服务管理单元。
Nginx 配置	<code>server/deploy/nginx/hertz-track.conf</code>	Nginx 反向代理配置示例。
二进制部署脚本	<code>server/deploy/scripts/deploy_binary.sh</code>	自动化二进制部署的示例脚本。
Makefile	<code>server/Makefile</code>	提供了 <code>docker-build</code> , <code>compose-up</code> 等快捷命令。